

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 4

The xv6 Shell from the Outside

Track B: Build Your Own Small OS (xv6)

Elaborated by:

st. gr. FAF-241

Ermicev Sofia

Verified by:

Patricia Reîțman

Contents

1	Introduction	2
2	Part 1. Fixing and Testing sleep	2
2.1	System call used by the current xv6 tree	2
2.2	Observation	3
3	Part 2. Bash versus the xv6 Shell	3
3.1	Selected Bash output	4
3.2	Selected xv6 output	5
3.3	Observations	5
4	Part 3. Who Expands *? Inspecting argc and argv	6
4.1	xv6 version of args	6
4.2	Results in Bash	7
4.3	Results in xv6	7
4.4	Observations	8
5	Part 4. Reading the xv6 Shell Source	8
5.1	Observations	9
6	Conclusion	10

1. Introduction

The purpose of this laboratory work is to investigate the xv6 command shell from the user's point of view and compare its behavior with Bash on Linux. The work demonstrates which command-line features are implemented by a small educational shell and which conveniences are added by a modern shell such as Bash.

The laboratory also examines how command-line arguments are constructed before a program starts. A small args program was implemented for both xv6 and Linux to show the contents of argc and argv. Finally, the source code of user/sh.c was inspected to connect the observed behavior with the internal command types implemented by xv6.

The experiments were performed in Ubuntu under WSL2 and in the current MIT xv6-riscv tree.

2. Part 1. Fixing and Testing sleep

2.1. System call used by the current xv6 tree

The current user API was inspected with:

```
grep -n "pause\|sleep" user/user.h
```

The result was:

Listing 1: The current xv6 user API

```
25:int pause(int);
```

Therefore, this xv6 version uses `pause(int)` instead of the older `sleep(int)` system call.

The implemented user/sleep.c was:

Listing 2: sleep.c adapted to the current xv6 API

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
}
```

```
}

pause(atoi(argv[1]));
exit(0);
}
```

The Makefile contained the program in UPROGS:

```
$U/_sleep\
```

After running `make qemu`, xv6 booted successfully and the program was tested:

Listing 3: Testing sleep in xv6

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

2.2. Observation

The program waits for the requested number of ticks when exactly one argument is supplied. If the argument is missing, it prints a usage message and exits with an error. The change from `sleep()` to `pause()` is an API change in the current xv6 tree; the user program itself still presents the command name `sleep`.

3. Part 2. Bash versus the xv6 Shell

The same command-line features were tested in Bash and in the xv6 shell. A Linux test directory contained a file named `README`. The xv6 root directory already contained its own `README` and user programs.

Command / feature	Bash	xv6 shell
echo hello > f ; cat < f ls grep READ	Printed hello; redirection and sequencing worked. Printed README.	Printed hello; redirection and sequencing worked. Printed the xv6 README en- try.
(echo a; echo b) wc	Printed 2 2 4.	Printed 2 2 4.
echo one ; echo two	Printed one and two.	Printed one and two.
echo bg &	Ran in background and Bash displayed a job number/PID.	Ran in background, but without Bash job-control information.
ls *	Bash expanded * to file- names before running ls.	Failed with ls: cannot open *; the * remained literal.
echo \$HOME echo "a b"	Printed /home/sofia. Printed a b; quotes were re- moved by the shell.	Printed literal \$HOME. Printed "a b"; quotes were not interpreted as Bash quoting syntax.
cd d followed by ls	ls remains available through PATH.	ls failed because xv6 does not search PATH. ../ls worked.
History / completion	Arrow-up, Tab completion and Ctrl+R are available.	No Bash-style history search or tab completion is imple- mented.

3.1. Selected Bash output

Listing 4: Selected results in Bash

```
hello
README
    2      2      4
one
two
[1] 2558
bg
README f
/home/sofia
a b
README d f
```

3.2. Selected xv6 output

Listing 5: Selected results in the xv6 shell

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

After changing to directory d, an ordinary `ls` could no longer be executed:

Listing 6: Running a program after changing directory in xv6

```
$ cd d
$ ls
exec ls failed
$ ../ls
.          1 1 1024
..         1 1 1024
README    2 2 2441
...
ls         2 11 41520
...
d          1 26 32
```

3.3. Observations

1. Both shells support execution of programs, input/output redirection, pipes, command lists/sequences, grouping with parentheses, and background

execution. Bash additionally provides pathname expansion (globbing), environment-variable expansion, quote processing, PATH lookup, interactive history, reverse history search, completion, and richer job control.

2. In xv6, `ls` stops working after `cd d` because the shell does not search directories listed in a `PATH` variable. The xv6 user programs are files in the parent/root directory, so from inside `d` the program can be executed explicitly as `../ls`.
3. For daily work, `PATH` lookup and command history/completion are among the most useful missing conveniences. Of the missing features, a simple `PATH` search would be comparatively straightforward to add because the shell could try a small list of directories before reporting exec failure; full quoting and expansion semantics would require a more substantial parser.

4. Part 3. Who Expands *? Inspecting `argc` and `argv`

4.1. xv6 version of args

A small xv6 program was created to print every argument it receives:

Listing 7: `user/args.c`

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++){
        printf("argv[%d] = [%s]\n", i, argv[i]);
    }
    exit(0);
}
```

It was added to `UPROGS`:

```
$U/_sleep\
$U/_args\
$U/_forktest\
```

A matching Linux program, `args_linux.c`, used `stdio.h` and returned 0 from `main`. It was compiled using:

```
gcc args_linux.c -o args
```

4.2. Results in Bash

The Linux test directory contained `args_linux.c`, `one.c`, and `two.c`. The observed output was:

Listing 8: `argv` produced through Bash

```
$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [one.c]
argv[3] = [two.c]
argv[4] = [x y]

$ ./args $HOME
argc = 2
argv[1] = [/home/sofia]

$ ./args 'single quoted'
argc = 2
argv[1] = [single quoted]
```

4.3. Results in xv6

The same tests in `xv6` produced:

Listing 9: `argv` produced through the `xv6` shell

```
$ args one two three
argc = 4
argv[0] = [args]
```

```

argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]

```

The single-quote experiment likewise showed that xv6 does not implement Bash-style quote processing: quote characters are treated as ordinary characters and whitespace still separates words.

4.4. Observations

1. For `*.c "x y"`, the Linux program received **5 arguments in total**: the program name, three filenames produced by glob expansion, and one argument `x y`. In xv6, the program received **4 arguments in total**: the program name, literal `*.c`, `"x`, and `y"`.
2. The program itself and the kernel do not perform the expansions demonstrated here. The **shell** processes the command line before calling `exec`. Bash expands `*.c`, substitutes variables, removes syntactic quotes, and constructs the final argument vector. The xv6 shell implements far fewer transformations, so the program receives the special characters literally.
3. Extra spaces in `one two three` do not create empty arguments. Both shells tokenize on whitespace and pass the three words as three arguments, giving `argc = 4` including the program name.

5. Part 4. Reading the xv6 Shell Source

The shell implementation was inspected directly in `user/sh.c`.

```

wc -l user/sh.c
grep -n "#define" user/sh.c

```

```
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```

The file contained **499 lines**. The command kinds were defined as:

Listing 10: Command kinds defined in user/sh.c

```
8:#define EXEC 1
9:#define REDIR 2
10:#define PIPE 3
11:#define LIST 4
12:#define BACK 5
14:#define MAXARGS 10
```

The execution switch contained corresponding cases at lines 75, 83, 93, 101 and 125:

Listing 11: Where the command kinds are executed

```
75: case EXEC:
83: case REDIR:
93: case LIST:
101: case PIPE:
125: case BACK:
```

The source also showed cd handling near line 170:

Listing 12: Location of cd handling

```
170:      fprintf(2, "cannot cd %s\n", cmd + 3);
```

5.1. Observations

1. The five command kinds are:

- EXEC – a normal command with no special operator;
- REDIR – produced by input/output redirection operators such as <, > and >>;
- PIPE – produced by |;
- LIST – produced by ;;
- BACK – produced by &.

Parentheses are parsed to group a subcommand, but they do not introduce a sixth command kind.

2. `cd` is handled specially by the shell's main loop rather than being executed as a normal child program. It must modify the current shell process's working directory. If `cd` were executed only in a forked child, the parent shell would remain in its original directory after the child exited.
3. No shell-expansion implementation for Bash-style `*` globbing or `$` variable substitution is present in `sh.c`. Occurrences of `*` found by a simple `grep` are primarily C pointer syntax, not wildcard handling. This explains why `ls *` passes a literal asterisk and why `echo $HOME` prints the variable name unchanged in Part 2.

6. Conclusion

This laboratory work demonstrated that even the small `xv6` shell implements the essential mechanisms of a Unix command interpreter: program execution, redirection, pipes, command lists, grouped commands, background execution, and a built-in `cd` operation.

The comparison with Bash showed that many features commonly associated with “running a command” are actually services provided by the shell before the target program starts. Bash performs wildcard expansion, variable substitution, quote processing, `PATH` lookup, history and completion, while `xv6` intentionally omits these conveniences.

The `args` experiments made this separation especially clear. Programs receive an already constructed `argv`; in Bash, the shell transforms `*.c`, variables and quoted text before `exec`, whereas the `xv6` shell passes most of these characters literally. Reading `user/sh.c` connected these external observations with its five internal command types and showed how a compact shell can provide the core Unix command-line model in about 500 lines of C code.